

Features — Phase 2

Gap analysis of requirements-vers-2.md (PRD v2.0, 11 Aug 2026) against the code as built.

Written 13 Aug 2026. v2 reframes the product from a single-municipality booking app into a multi-tenant recreation platform with six modules. What we shipped (Phases 1–6 + §4.9–4.11 extras) is essentially **one** of those modules — “Facility and asset booking” — plus a partial “Financial operations” and “Administration”.

Every “what we have” claim below was verified against the code, not against CLAUDE.md.

0. How to read this

Verdict	Meaning
DONE	v2 requirement is already met; no phase-2 work.
PARTIAL	Something exists but falls short of v2’s acceptance criteria. Scope listed.
MISSING	Nothing in the codebase addresses it.
PROCESS	Not a code deliverable — a commercial, hosting or documentation commitment.

Section numbers match requirements-vers-2.md so the two documents can be read side by side.

1. Summary

v2 §	Requirement	Verdict	Phase-2 work
4	Platform and tenancy	MISSING	Tenant isolation, branding, terminology, module licensing, config promotion + versioning

v2 §	Requirement	Verdict	Phase-2 work
5.1	Asset directory — hierarchy, sub-spaces	PARTIAL	Site/building/space/sub-space model; station division; hierarchy-aware conflict detection ; pagination
5.2	Availability — rollover, mass update	PARTIAL	Seasonal rollover with preview; mass update with reverse
5.3	Search by parameters	PARTIAL	Cost-range, location/area, accessibility filters; guided empty state
5.4	Search by date and time	DONE	—
5.5	Booking and approval	PARTIAL	Priority/allocation windows; cross-hierarchy concurrency; series conflict preview; approval conditions
5.6	Calendar sync	PARTIAL	Two-way M365 / Google Workspace sync (see 7.4)
5.7	Programs and registration	MISSING	Entire module
5.8	Memberships	MISSING	Entire module
5.9	Admissions, retail and POS	MISSING	Entire module
5.10	Events, permits and workflow	MISSING	Entire module (only waiver upload exists)
5.11	Financial operations	PARTIAL	Fee engine, rate blending, tax, invoicing, GL coding + export, reconciliation, report builder

v2 §	Requirement	Verdict	Phase-2 work
5.12	Staff back-office	PARTIAL	Department-scoped permissions, bulk import/edit, staff dashboard, training environment
5.13	My account	PARTIAL	Unify registrations, memberships, permits, counter history
5.14	Notifications	PARTIAL	Real email delivery, SMS, per-tenant templates
5.15	Further capabilities	PARTIAL	Website embedding; demand reporting (map, waitlist, public view, bilingual all DONE)
6	Data model	PARTIAL	8 new entities + 4 expanded (detailed in §P2-6)
7.1	Accessibility	PARTIAL	Contrast audit, mobile pass, tablet-usable staff UI, ACR document
7.2	Security and privacy	PARTIAL	Tenant-configurable SSO + MFA, PCI-DSS, residency, MFIPPA/FOIP, DR/RPO/RTO
7.3	Performance and scale	MISSING	Pagination, indexing, caching, horizontal scale, load-test evidence

v2 §	Requirement	Verdict	Phase-2 work
7.4	Integration and portability	MISSING	Versioned public API, migration tooling, data export, GL + calendar integrations
7.5	Service operations	PROCESS	SLA, support model, maintenance windows, roadmap

Six modules’ worth of new build. Realistically this is not one phase; §P2-9 proposes a split.

P2-4. Platform and tenancy — MISSING

v2 §4 is entirely new, and it is the change with the widest blast radius: it touches every query in the codebase.

What we have. A single-tenant application. Configuration is process-level environment variables (`internal/config/config.go`, ~30 `FB_*` keys) read once at boot. Branding is hardcoded in the SPA (“Rivermont Spaces”). No **Tenant** entity; no tenant column on any model (`internal/domain/base.go` gives every row a UUID, timestamps and soft-delete — nothing more).

Missing.

- **Tenant** entity + a **TenantID** on every tenant-scoped model, with query scoping enforced in a single place (a GORM scope or callback on **Base**) rather than per-service — this is the only way it stays correct as services multiply.
- Tenant resolution from the request: custom domain or subdomain → tenant, injected into request context ahead of every handler.
- Per-tenant branding: logo, colour tokens, portal domain. The SPA currently has a fixed Tailwind theme; needs runtime theming from a tenant config endpoint.
- Per-tenant terminology overrides (a slip, a berth, a diamond, a pad). Note this interacts with i18n: `web/src/lib/i18n.ts` holds static EN/FR bundles, so term overrides must layer on top of the `t()` lookups rather than replace them.
- Module licensing: an enabled-modules set per tenant, enforced **server-side** (routes refuse when the module is off) and reflected in SPA navigation.

Server-side is not optional — hiding a nav link is not access control.

- Separate production / staging / training environments per tenant, with configuration promoted between them without a deployment.
- Versioned, attributable, rollback-able configuration changes. Our `domain.AuditLog` + `internal/auditlog` covers *booking* actions; config changes are not captured at all.

Acceptance criteria to satisfy: all three in v2 §4 — differing fee rules with no code difference; booking-only tenant sees no registration; staging→production promotion with no release.

Note. Multi-tenancy is far cheaper to retrofit now (10 domain models) than after the programme/membership/POS modules land (25+). If phase 2 does anything, it should do this first.

P2-5.1. Asset directory — PARTIAL

What we have. `domain.Facility` with name, description, capacity, fee, deposit, location, image, lat/long, before/after instructions, `StepFreeAccess`, `AccessibleWashroom`, and `FacilityAccessory` join rows carrying quantities. A self-referential `ParentID *string` exists. Directory + detail pages in `web/src/pages/FacilityList.tsx` / `FacilityDetail.tsx`.

Missing.

1. **Hierarchy semantics.** `ParentID` is stored but carries no meaning: nothing distinguishes site / building / space / sub-space, and there is no flag for “parent is bookable in its own right” vs “bookable only through its children”. Needs an `AssetKind` (or level) plus `BookableDirectly bool`.
2. **Station division** (pool → lanes, gym → courts, hall → halves). Nothing models this.
3. **Hierarchy-aware conflict detection — a correctness gap, not just a missing feature.** `booking.loadWindow(internal/booking/service.go:98)` scopes its blackout and overlap queries to a single `facility_id`:

`Where("facility_id = ? AND status IN ? AND ends_at > ? AND starts_at < ?", facilityID,`

So today, booking court 2 does **not** block the whole-gymnasium asset, and booking the gymnasium does not block its courts. v2 §5.5 requires the block to hold “across parent and child assets”, and v2 §5.1’s acceptance criterion (“court 2 booked → courts 1 and 3 available, whole-gymnasium unavailable”) fails as built. Fix: resolve the ancestor+descendant closure of the target asset inside the transaction and scan bookings across that whole set, keeping the `SELECT ... FOR UPDATE` lock. Availability rendering (`internal/availability, facility.Calendar`) needs the same closure or the calendar will show open slots the booking endpoint then rejects.

4. **Floor plans** — only a single `ImageURL`; v2 wants photos (plural) and floor plans.
5. **Fuller accessibility details** — v2 names accessible parking and hearing loop; we have two booleans.
6. **SCALE: 8,000+ assets across 20+ facilities.** `facility.Service.List` (`internal/facility/service.go:33`) returns *every* matching row — no `LIMIT`, no cursor, no count. It preloads `Accessories.Accessory` for all of them, and `FacilityList.tsx` renders the lot. At 8,000 assets this is a slow query and a multi-megabyte JSON payload. Needs pagination (API + UI), server-side sort, and indexes for the filter columns.

Also DONE: free assets already display “Free” explicitly (`FeeCents == 0`), satisfying that acceptance criterion.

P2-5.2. Availability and scheduling — PARTIAL

What we have. `domain.AvailabilityRule` (per-weekday open/close minutes), `domain.Blackout` (date-range closures with a reason), min/max duration and buffer minutes on the facility, per-day slot generation in `internal/availability`, and a public weekly/monthly calendar (`GET /api/facilities/{id}/calendar` → `web/src/pages/FacilityCalendar.tsx`) that distinguishes open / booked / blackout. Staff manage blackouts via `/api/staff/facilities/{id}/blackouts`.

Missing.

- **No season concept at all.** Rules are open-ended weekday patterns with no effective-date range, so “roll a season forward” has nothing to roll. Needs a **Season** (or effective-dated rule sets) before rollover can exist.
 - **Seasonal rollover** across ~500 assets in one operation, carrying schedules, rules *and* rates forward, with a **preview of what will change before commit**.
 - **Mass update tooling** across a selected asset set, **reversible** — which implies a recorded change-set with a stored before-state, not just a bulk `UPDATE`.
-

P2-5.3. Search by parameters — PARTIAL

What we have. `facility.Filter` (`internal/facility/service.go:26`) supports exactly three constraints: `MinCapacity`, `FreeOnly`, and a single `Accessory` name — exposed as `?minCapacity=&free=&accessory=`.

Missing.

- **Cost range.** Only a free-only boolean; v2 asks for a range.
- **Location or area.** Location is free text with no area/zone dimension to filter on.
- **Accessibility needs** as a filter (the fields exist on the model but aren't filterable).
- **Multiple required amenities** — the filter takes one accessory name, not a set.
- **Guided empty state** suggesting which filter to relax. Today an empty result renders as an empty list.

Small, cheap, and directly resident-facing — good early phase-2 work.

P2-5.4. Search by date and time — DONE

?date=&start=&end= ANDs with the parameter filters and returns only assets free for the whole window (`internal/httpapi/facilities.go:29`, `facility.Service.Search`). Meets v2 §5.4.

P2-5.5. Booking request and approval — PARTIAL

What we have. Booking captures asset, window, purpose, attendance and residency-at-booking. Auto-confirm vs `RequiresApproval` per facility. Pending → confirmed / denied / cancelled with notification at each step, staff approve/deny/refund, cancel, reschedule (§4.9). Double-booking is prevented inside a `Transaction` with `clause.Locking{Strength: "UPDATE"}` on the overlap scan (`internal/booking/service.go:47,110`) — genuine row-level locking, with a concurrency test in `internal/booking/service_test.go`. Weekly recurring bookings grouped by `RecurrenceID`, skipping conflicting weeks.

Missing.

- **Concurrency across the hierarchy** — the lock is per-`facility_id` only. See P2-5.1 item 3. The v2 concurrency criterion passes for a single asset and fails for parent/child.
- **Allocation priority windows.** No concept of an affiliated/resident-only booking period before general release, and no “general booking opens on ” message. Needs a per-asset (or per-season) window with an eligibility rule, enforced server-side.
- **Recurring/seasonal as a single application with conflicts reported before submission.** Ours silently *skips* conflicting weeks at submission time (`RequestRecurring`, `internal/booking/service.go:182`); v2 wants the conflicts surfaced for the applicant to resolve first, and the series treated as one application.

- **Approval carrying conditions, fees or required documents.** Approval is a plain status transition. `RequiresWaiver` is the only document gate, and it is a facility-level flag set before booking, not a condition a staff member attaches at approval time.
- Attendance is captured but not validated against capacity.

P2-5.6. Calendar sync and invitations — PARTIAL

What we have. One-way `.ics` invite to the booker (`GET /api/bookings/{id}/invite.ics`) and a public read-only iCal feed for the city (`GET /api/calendar.ics`), both in `internal/calendar`. Cancel and reschedule re-issue the invite.

Missing. Real sync to the municipality’s calendar platform — Microsoft 365 and Google Workspace, named in v2 §7.4. An iCal feed is a poll, not a sync: it cannot withdraw an entry, and subscriber refresh is client-controlled. Needs OAuth per tenant, event create/update/delete, and an external-id mapping (the `IntegrationRecord` entity in v2 §6).

P2-5.7. Programs and registration — MISSING (entire module)

Nothing in the codebase touches this. Edmonton: ~20,000 courses, 100,000+ registrations/year.

To build.

- **Programme / session / class** definitions with seasons, schedules and repeating patterns, built from **reusable templates** (staff will not hand-enter 20,000 courses).
- Configurable registration open/close dates, with optional resident-priority windows (shares machinery with P2-5.5 allocation windows).
- Eligibility rules: age, date of birth, skill level, completed prerequisite. Note DOB is new personal data — `domain.User` holds only email, name, role, residency and address today.
- **Progression tracking** through skill levels; completion advances the participant and satisfies the next level’s prerequisite. v2 §8 Q7 flags whether level schemes are municipality-defined or fixed — **this is a blocking design decision**, and municipality-defined is the answer consistent with v2’s “rules belong in configuration” rule.
- Capacity limits with automatic waitlists and **automatic promotion** when a place is released. Our `internal/waitlist` is booking-specific and notify-only (it emails waiting residents when a slot frees); programme waitlists must *offer and hold* the place.

- Participant records with emergency contacts and medical/accessibility notes, **visible only to authorised staff** — the most sensitive data in the platform, and the first thing here needing field-level access control.
- Instructor assignment, attendance tracking, **mobile-usable rosters**.
- Withdrawals and transfers between sessions with policy-driven refunds or account credits (account credit is a new financial primitive — see P2-5.11).
- **SCALE: 100+ registrations/second at opening with no oversell and no lost registrations.** Our proven pattern (`SELECT ... FOR UPDATE` inside a transaction) is correct but serialises on the hot row; 500 simultaneous attempts on one 100-place course is the exact worst case for it. Needs a queue/reservation design, and the default SQLite driver cannot serve it at all.

Acceptance criteria: prerequisite refusal with explanation; automatic waitlist promotion on withdrawal; 500 attempts on 100 places → exactly 100 succeed, rest waitlisted, none lost.

P2-5.8. Memberships — MISSING (entire module)

Nothing in the codebase. Edmonton: 130,000+ members across 20+ facilities.

To build.

- Membership products: term, price, entitlements, and which facilities and times they admit.
- Categories — individual, family, corporate, subsidised — each with own pricing and eligibility.
- **Fee-assistance / low-income subsidy** programmes with eligibility verification and **discreet handling at point of sale** (the subsidised rate must apply without disclosing status to bystanders — a UI requirement, not just a data one).
- One membership valid across multiple facilities, with access rules evaluated **per facility and per time of day**.
- Lifecycle: join, renew, freeze, transfer, cancel, with pro-rating.
- Entitlement checks applied automatically when a member books a space or registers for a programme — i.e. the booking and registration paths both call into the membership engine.
- **Household / named dependants** (v2 §6 expands `Customer` to carry household members). We have a flat `domain.User`; a family membership covering four named dependants has nowhere to live.

Acceptance criteria: off-peak membership refused at peak with drop-in fee offered; fifth person refused under a four-dependant family membership; subsidised rate applied discreetly.

P2-5.9. Admissions, retail and point of sale — MISSING (entire module)

Nothing in the codebase — and v1 assumed all payment happens on a web form (`internal/payment` is a mock-Stripe web checkout behind a `PaymentProvider` interface).

To build.

- POS for counter transactions **with payment terminal hardware** and a published supported-device list. Hardware integration is the single largest unknown in phase 2.
- Drop-in admission with fast credential validation — **granted or refused within two seconds** at a busy queue.
- **Revenue splitting** across departments/partners within one transaction.
- Merchandise inventory: stock levels, receiving, adjustments, retail sale through the same till.
- Cash handling: float management, till reconciliation, daily cash-out with variance recorded.
- Counter transactions attach to the **same customer account** as online activity (feeds P2-5.13).

Note. v2 §8 Q2 asks build / partner / defer, and calls it out as undeferrable because it is hardware-dependent. It is also the module least served by our current architecture: everything we have assumes a browser session and an authenticated resident. **Recommend answering Q2 before any phase-2 estimate is committed.**

P2-5.10. Events, permits and workflow — MISSING (module in all but name)

What we have. Only the document-upload primitive: `internal/media` (magic-byte sniffing, type whitelist, image re-encode, size/decompression caps, random storage name, stored outside any web root, served with `nosniff` + locking CSP + attachment) and `internal/waiver` gating booking confirmation on a signed waiver. That is a good foundation for permit document upload and should be reused as-is.

Missing. Everything else:

- **Configurable application forms per permit/event type, built by staff without code** — a form builder plus dynamic form rendering and storage. No form-definition entity exists.
- **Multi-stage approval workflow** routed by department, application type or value, including **parallel approvals** across departments. Our approval is one staff action on one booking.

- **Document expiry tracking and renewal reminders.** `domain.WaiverDocument` stores only `BookingID`, stored name, content type and size — no issue date, no expiry, no reminder state. (`internal/reminders` already has the scheduler pattern to copy — it nudges bookers within 24h, idempotent via `Booking.ReminderSentAt`.)
- **Applicant-visible status tracking with internal staff notes kept separate** from applicant correspondence.
- **Conditions attached to an approval, acknowledged by the applicant before the permit issues.**
- **Post-event invoicing for actual costs**, reconciled against the permit and the original estimate.
- **Configurable service standards with escalation** when an application stalls past target.

Acceptance criteria: three-department approval identifies the outstanding department and withholds the permit; unacknowledged conditions withhold the permit; over-estimate actual costs generate a reconciled invoice.

P2-5.11. Financial operations — PARTIAL

What we have. Fee and deposit shown before commit; mock-Stripe checkout behind a `PaymentProvider` interface (`internal/payment`) with a `StripeProvider` slot ready; `domain.Payment` (one current row per booking) plus an append-only `domain.PaymentTransaction` ledger recording every charge, decline and refund with provider ref and card last-4; staff refunds (`POST /api/staff/bookings/{id}/refund`) and a payments ledger screen (`StaffPayments.tsx`); resident vs non-resident pricing (`Facility.FeeFor`); deposits stored (`DepositCents`); a reporting dashboard (`internal/reports` → `StaffReports.tsx`) with revenue, bookings, utilisation, pending/overdue, by-facility, top spaces, 6-month trend and resident split.

Missing.

- **A fee engine.** Pricing today is two integers on the facility — `FeeCents` and `NonResidentFeeCents` — a flat per-booking charge. v2 needs rates by asset, **time of day**, **day of week**, **season**, **duration** and **customer category** (non-profit, commercial). This is the `FeeSchedule` entity, and it must be shared by bookings, registrations, memberships and POS.
- **Rate blending.** A 4pm–8pm booking crossing a 6pm rate change must charge each portion at its own rate. Nothing today can express this — the fee is a single lookup.
- **Tax configuration per fee type with exemptions.** No tax handling anywhere; stored amounts are single `AmountCents` values.
- **Invoicing** for accounts billed after the fact, with payment terms and aging. Payment is pay-now only.

- **GL coding per fee** and an export/interface to the municipality’s financial system. No GL concept exists.
- **Daily reconciliation and revenue reporting with CSV and Excel export.** Verified: there is no export path in `internal/reports` — the dashboard is JSON to screen only. v2 wants each export line to carry its GL code and to balance to the day’s takings.
- **An ad-hoc report builder staff can use without vendor involvement.** Our reports are hand-written SQL aggregations with a fixed month/quarter/year period.
- **Deposit hold and release** as a lifecycle. `DepositCents` is stored but there is no hold/release flow.
- **Account credits** (needed by programme withdrawals, P2-5.7).
- **Transaction expansion** (v2 §6): revenue splits, channel (online vs counter), GL posting.
- Refunds are staff-initiated; v2 wants policy-driven automatic refunds on cancellation within the refundable window, with the reversal reflected in the GL export.
- **REVISED, and already satisfied:** no patron-facing convenience surcharge — we charge none.

At \$70M/year of transactions (Edmonton), this module carries the most audit exposure of anything in phase 2.

P2-5.12. Staff back-office — PARTIAL

What we have. Facility create/edit/delete + blackout management (`StaffFacilities.tsx`), approval queue (`StaffQueue.tsx`), payments ledger, reports, user/role admin (`StaffUsers.tsx`), audit log viewer (`StaffAudit.tsx`). Three global roles — resident / staff / admin (`domain.Role`) — enforced by `auth.RequireRole` middleware in `internal/httpapi/router.go:109,127`.

Missing.

- **Department-scoped permissions.** Roles are global: any `staff` user can approve any booking and edit any facility. v2 requires staff to see only their own service area — needs a department dimension on users and assets, and scoping in the service layer (not just the UI).
- **Bulk import and edit** for assets, customers, schedules and programmes — with validation and an error report, not a silent partial import.
- **A staff dashboard** summarising pending approvals, today’s bookings and exceptions needing attention. We have a pending-approvals queue and an analytics dashboard; neither is the at-a-glance operational view v2 describes.
- **A training environment with representative data.** Our `internal/seed` package already generates a deterministic year of history

(7 facilities, ~600 bookings, payments) — good raw material, but there is no separate training tenant/environment to point it at (ties to P2-4).

- No cross-asset master schedule view for staff.

P2-5.13. My account — PARTIAL

What we have. `MyBookings.tsx` + `/api/bookings/mine` and `/api/waitlist/mine`: bookings and waitlist entries, with self-service cancel and reschedule.

Missing. The unified account v2 describes — programme registrations, memberships, permits and **payment history across both online and counter channels** in one place. Blocked on P2-5.7, 5.8, 5.9, 5.10. Cancellation/modification policy windows are also hardcoded (owner may change an active booking before it starts) rather than configurable per tenant.

P2-5.14. Notifications and reminders — PARTIAL

What we have. `notify.Notifier` interface with **one implementation: `LogNotifier`** (`internal/notify/notify.go:30`) — it writes to the log. Verified: there is no SMTP client and no `FB_SMTP_*` config. Reminder scheduling works (`internal/reminders`, idempotent via `Booking.ReminderSentAt`) and reminders carry access instructions, as v2 requires. Booking submitted / confirmed / denied / cancelled / reminder / waitlist-opened events all fire.

Missing.

- **Actual email delivery.** For a sales demo the log notifier is fine; for v2 it is the gap between “notifies” and “notified”.
- **SMS as a configurable option.**
- **Per-tenant configurable templates** with municipal wording and branding (ties to P2-4).

P2-5.15. Further capabilities

Capability	Verdict
Map view of assets + directions	DONE — <code>web/src/pages/MapView.tsx</code> , Leaflet + OSM, facility lat/long
Waitlists on popular assets with notification	DONE — <code>internal/waitlist</code>
Public read-only availability without an account	DONE — <code>/availability</code> , <code>/api/facilities/{id}/calendar</code>

Capability	Verdict
Utilisation and revenue reporting Demand reporting	DONE — <code>internal/reports</code> PARTIAL — utilisation and revenue only; nothing measures unmet demand (searches with no result, waitlist depth, denied requests)
Bilingual EN/FR	DONE (UI) — <code>react-i18next</code> , <code>web/src/lib/i18n.ts</code> , every page via <code>t()</code> . Facility content is single-language : names, descriptions and instructions are one <code>varchar/text</code> per field. A real bilingual deployment needs translated content, which is a schema change.
Bilingual selectable per tenant Website embedding of availability and registration	MISSING — ties to P2-4 MISSING — needs embeddable widgets, a framing policy that isn't <code>X-Frame-Options: DENY</code> (currently set globally in <code>internal/httpapi/security.go</code>), and a CSP <code>frame-ancestors</code> allowlist per tenant

P2-6. Data model deltas

New entities from v2 §6, none of which exist today:

Entity	Notes
Tenant	Configuration, branding, terminology, enabled modules
Programme	Course, session, schedule, capacity, eligibility, prerequisites, instructor
Registration	Participant, programme, status, attendance, progression, waitlist position
Membership	Product, category, term, entitlements, access rules, subsidy status
PermitApplication	Form data, documents, approval stages, conditions, decisions, costs
FeeSchedule	Rate rules by asset/time/season/category; tax treatment; GL codes

Entity	Notes
<code>InventoryItem</code>	Merchandise, stock level, receiving and adjustment history
<code>IntegrationRecord</code>	External identifiers linking bookings, calendars and financial postings

Expanded existing entities:

Entity	Expansion needed
<code>Facility</code> → <code>Asset</code>	Hierarchy level + parent semantics, station division, <code>BookableDirectly</code> , richer accessibility, multiple media, floor plans
<code>User</code> → <code>Customer</code>	Household / named dependants, DOB, unified cross-channel history; keep the C2 sub link
<code>Payment</code> / <code>PaymentTransaction</code>	Revenue splits, channel (online/counter), GL posting, tax lines, deposits held/released, account credits
<code>User</code> and <code>role</code>	Department scope on staff accounts
<code>AuditLog</code>	Extend beyond bookings to permits, refunds and configuration changes
(all tenant-scoped models)	<code>TenantID</code> + enforced query scoping

Naming. v2 consistently says *asset*, not *facility*, and *customer*, not *user*. Renaming `domain.Facility` → `domain.Asset` is a wide but mechanical change; doing it at the start of phase 2 is cheaper than living with two vocabularies across six modules.

P2-7. Non-functional

7.1 Accessibility — PARTIAL

Have: `<html lang>` synced to the EN/FR toggle, skip-to-content link, `main/nav` landmarks, focus-visible rings on interactive controls, `role="alert"/status` live regions on errors and spinners, accessible names on every control (verified: no unnamed buttons/inputs, no images without alt), slate-500 floor on informational text.

Missing: full WCAG 2.1 AA contrast audit; mobile pass; **staff functions usable on a tablet** (the staff tables are desktop-shaped today); AODA + provincial-equivalent alignment; a **published accessibility conformance report** available on request.

7.2 Security and privacy — PARTIAL

Have: OIDC login against C2 with local sessions and roles; HTTP-only session cookies; server-side authorization via `RequireAuth/RequireRole`; security headers on every API response (`nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, `default-src 'none'` CSP, HSTS when `FB_ENV=prod`); hardened media upload (`internal/media`); an **append-only, tamper-evident hash-chained audit log** via the central audit-logging service (`internal/auditlog`) — which substantially answers v2’s immutable-audit requirement for booking actions.

Missing:

- **Tenant-configurable SSO.** Our identity provider is C2, wired via `FB_OIDC_*` process env (`internal/auth`). v2 needs staff SSO against *the municipality’s* IdP, i.e. per-tenant OIDC/SAML configuration.
- **MFA on all administrative accounts** — currently delegated entirely to C2 and unverified here.
- **PCI-DSS compliant payment handling with no card data stored.** We store `CardLast4` on `PaymentTransaction` (permitted, but needs a formal scope assessment) and the provider is a mock. Real compliance starts with answering v2 §8 Q5 (which processor).
- **Canadian data residency** for all customer data — a hosting decision (v2 §8 Q6); today the app deploys to a single shared host (`deploy.md`).
- **Named sub-processor list** with advance notice of change.
- **MFIPPA / FOIP compliance**, including **support for access and correction requests** — this is a feature (export/correct one customer’s data on request), not just a policy.
- **Encryption at rest** — not addressed; the default is a local SQLite file.
- **Annual third-party penetration testing** with a client-available summary.
- **Security incident detection and notification** within a defined period.
- **Documented backup, retention and DR with stated RPO/RTO.**
- Audit coverage must extend to permits, refunds and configuration changes.

7.3 Performance and scale — MISSING

v2’s numbers are architecture constraints: **10,000 concurrent users; 100+ transactions/second at registration peak; 8,000+ assets; 130,000+ member records** with no search or listing degradation.

Current state, concretely:

- **Default DB is a single-file pure-Go SQLite** (`FB_DB_DRIVER=sqlite`), chosen for portability. MariaDB is supported but is not the default. Neither is configured for horizontal scale.
- **No pagination anywhere.** `facility.Service.List` returns all matching rows with accessories preloaded; `bookings/mine`,

staff/bookings/pending and ListForCalendar are likewise unbounded.

- **Reports run full-table aggregations** per request (`internal/reports`) with no caching or pre-aggregation.
- **No caching layer, no read replicas, no queue.** Single process, single instance.
- The `FOR UPDATE` booking lock is correct but serialises on the contended asset — the wrong shape for a 100/second registration surge (see P2-5.7).
- **No load or soak test evidence**, which v2 §7.3 requires be available to prospective clients.

v2 helpfully notes these optimisations should be **deferred until a large client is genuinely in view** — but the cheap, non-deferrable parts are pagination, indexing and not defaulting to SQLite. Do those; defer the rest.

7.4 Integration and data portability — MISSING

- **Documented, versioned public REST API covering all core entities, treated as a product surface.** We have an internal JSON API for our own SPA: unversioned (`/api/...`, no `/v1`), no OpenAPI spec, no published documentation, and session-cookie auth only — no API keys or tokens for third-party callers. (CLAUDE.md’s mention of personal access tokens is aspirational; the router has none.)
- **Data migration tooling** to import customers, assets, bookings and history from a legacy system, **with validation and reconciliation reporting before cutover.**
- **Full customer data export in a non-proprietary format, on demand.**
- **Transition assistance on exit at published rates. No lock-in.** (PROCESS.)
- **Financial system integration** for GL postings and receivables (see P2-5.11).
- **Calendar integration with Microsoft 365 and Google Workspace** (see P2-5.6).

7.5 Service operations — PROCESS

SLA with a stated, measured uptime target; support model with severity levels and response and resolution targets; published maintenance windows; a version and patch register per component; a stated support lifecycle; published roadmap and release notes; an implementation methodology supporting phased delivery by service area alongside existing systems.

None of these are code, but every one is a deliverable a municipal procurement will score. The `/healthz` endpoint is the only piece of operational surface that exists today.

P2-8. Decisions that block estimating

v2 §8 raises eight open questions. Four of them gate phase-2 work and should be answered before committing dates:

1. **Q1 — Which municipality size is the design target?** A town of 20,000 and a city of 1,000,000 differ by two orders of magnitude. This decides how much of §7.3 is real work now versus deferred.
2. **Q2 — POS: build, partner or defer?** Hardware-dependent, mandatory for recreation buyers, and the module furthest from our current architecture. Undeferrable per v2.
3. **Q3 — Which modules are in the first release?** v2 suggests booking + financial operations is the smallest genuinely sellable combination. That matches our position: booking is largely built, so financial operations is the highest-leverage next module.
4. **Q7 — How is programme progression modelled?** Municipality-defined level schemes versus a fixed model changes the shape of the registration data model, and registration is the largest new module.

Q5 (payment processor and cloud region) and Q6 (Canadian residency in all cases) gate all of §7.2 — no security documentation can be written until they are answered.

P2-9. Suggested sequencing

Six modules plus a tenancy retrofit plus a scale story is not one phase. A defensible split, ordered so nothing has to be built twice:

Phase 2a — Foundations (do first, cheapest now, most expensive later). Multi-tenancy retrofit (P2-4) · **Facility** → **Asset** rename + hierarchy semantics · **hierarchy-aware conflict detection** (the outstanding correctness gap) · pagination and indexing (P2-7.3 cheap parts) · the remaining search filters (P2-5.3) · real email delivery (P2-5.14).

Phase 2b — Financial operations (v2 §8 Q3's other half of a sellable release). Fee engine + rate blending · tax · GL coding + export · invoicing and aging · deposits hold/release · account credits · CSV/Excel export + reconciliation.

Phase 2c — Programs and registration. The largest new module; needs 2a's tenancy and 2b's fee engine, and forces the scale question.

Phase 2d — Memberships. Depends on households (2a data model) and the fee engine (2b); entitlement checks then hook into booking and registration.

Phase 2e — Events and permits. Reuses **internal/media** and the audit trail; adds the form builder and multi-stage workflow.

Phase 2f — Admissions, retail and POS. Gated on v2 §8 Q2. Hardware-dependent and the least served by the current architecture.

Throughout: back-office depth (P2-5.12), accessibility completion (P2-7.1), the public versioned API and migration tooling (P2-7.4), and the §7.5 process deliverables.

One caveat worth stating plainly: the current build is a sales demo. Several of its deliberate demo shortcuts — mock payments, log-only notifications, default SQLite, seeded data — are exactly the things v2 turns into hard requirements. Phase 2 is not only additive; it also converts the demo's simulations into real integrations.